VEGA · AUTONOMOUS SYSTEM CONTROLLER

# The Control Loop

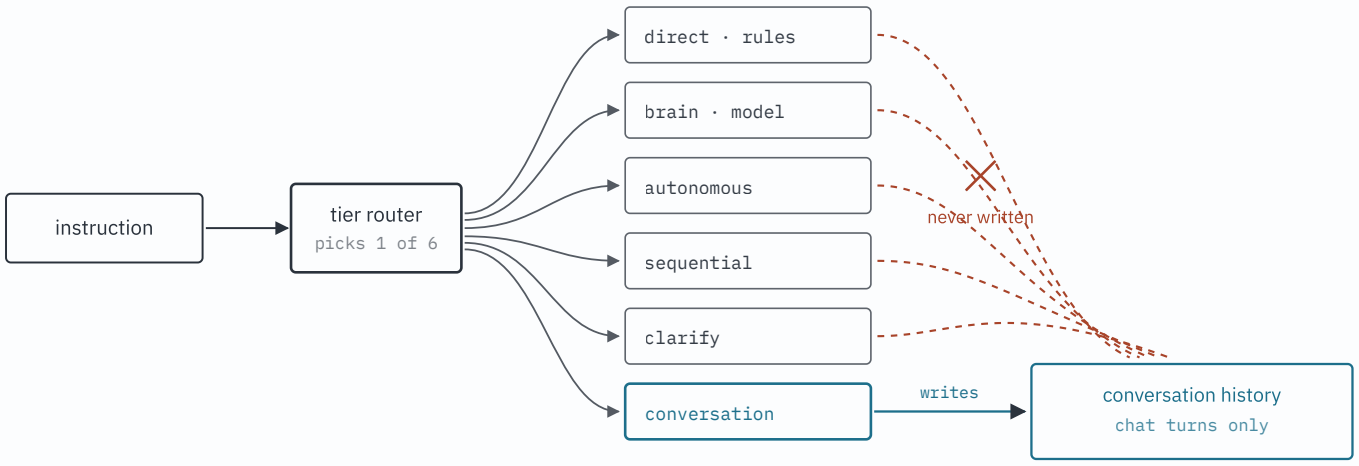
Current routing, its failure mode, and a proposed replacement.

FOR ARCHITECTURE REVIEW · SEPTEMBER 2026

VEGA is a local-first assistant that operates the machine it runs on — files, documents, browser, system, email, voice. It works, it is covered by 526 tests, and it has one structural problem this document exists to put in front of a reviewer: **the part that acts and the part that talks are separate systems, and neither knows what the other did.**

## 01 How a turn is handled today

A user instruction enters a router, which classifies it and hands it to one of six execution paths. Each path holds its own state and its own prompt. Whichever one runs, the others learn nothing.




> Five of the six paths perform real work — creating files, driving the browser, writing documents — and none of them writes to the history the conversation path reads. Asked afterwards where a file was saved, the conversational model is answering from a record in which nothing was ever saved.

OBSERVED IN TESTING

*“how many world heritage sites of maharashtra state creat report on it”* — routed to the analytics agent, executed in 2.0s, no web search, no report written, **recorded as success**. The follow-up *“where have you stored it”* then reached a model with no record that a report had ever existed, so it produced a plausible sentence instead of an answer.

Two distinct defects sit in that one exchange, and they need separating because they have different fixes.

- **Nothing verified the action was the right one.** The system confirms that an effect it performed actually happened — a file exists, a field holds the value, a process exited. It does not ask whether the action it chose was the one the instruction called for. Running the analytics agent succeeded; it just had nothing to do with world heritage sites.
- **Nothing carried the outcome forward.** Even a correct report would have been unanswerable afterwards, because execution results never reach the conversation.

## 02 Why the fast path has to go

The rules layer recognises about 95% of instructions without a model and dispatches them in under a millisecond. It exists for one reason: **a model call used to cost twelve seconds.**

That is no longer true. Three configuration defects were found and fixed this week — an environment variable that silently overrode config, an embedding model evicting the language model on every command, and a council design that swapped model weights between roles. Measured on the same machine, same model, no hardware change:

| MEASURE | BEFORE | AFTER |
|---|---|---|
| `model call` | 5.85 s | 0.27 s |
| `conversational reply` | 9.61 s | 0.63 s |
| `first instruction of a session` | 39 s | 0.94 s |

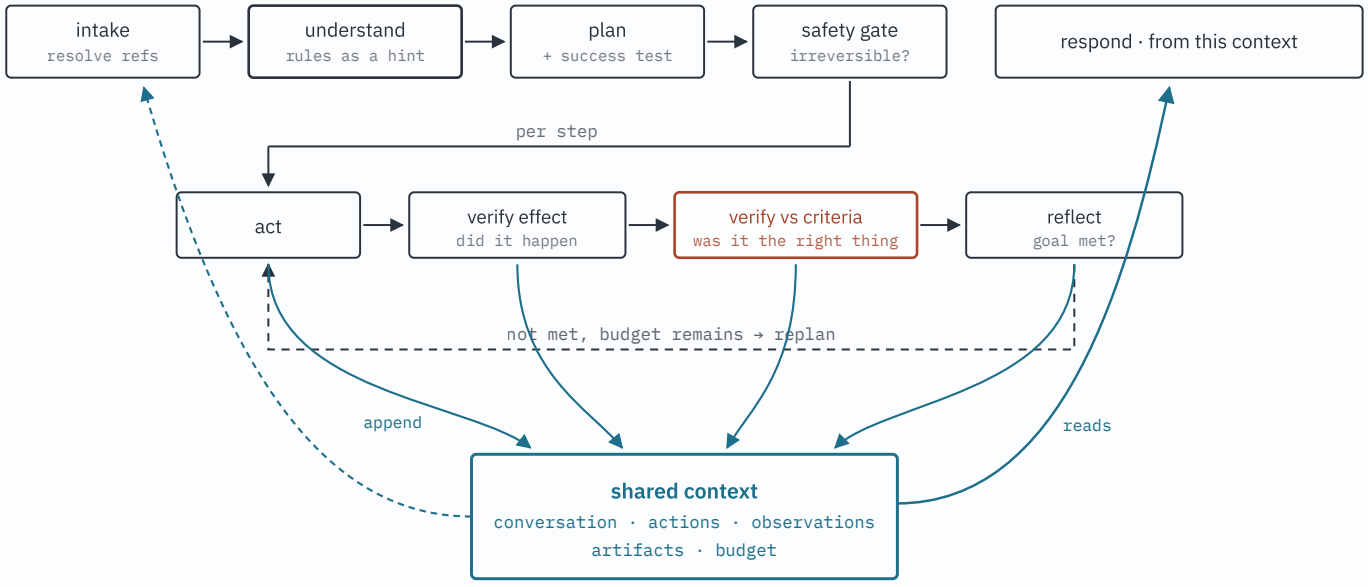
THE CONSEQUENCE

A model call is now **0.27 seconds**. Bypassing the model to save that is no longer a trade worth making — and every bypass is a turn the system took without understanding it. The fast path buys 0.27s and costs the system its awareness of its own actions.

The rules should not be deleted; they are accurate and they encode real knowledge. They change role: **from a shortcut around the model to a suggestion the model validates.** The prompt carries "the rules read this as `delete_tasks(title="call supplier")` " and the model confirms or corrects. Nearly as fast, and nothing happens that the system did not reason about.

## 03 Proposed: one loop, one context

Every turn enters the same loop and shares one context object. Conversation stops being a separate tier and becomes one of the things the loop can decide to do.




> Every stage writes into the same context, and the reply is generated from it — so “where did you save it” is answerable by construction rather than by passing a summary between two components. The red stage is the one that does not exist today: checking the result against what the step was supposed to achieve, not merely that the action ran.

### The stages, and what each is for

1. **Intake** — normalise, and resolve references against the context (*“that file”* → the path recorded three turns ago).

   Reference resolution is impossible today because no component holds both the conversation and the artifacts.

2. **Understand** — one model call over the full context and tool manifest. The rules layer contributes a suggestion; the learned classifier contributes a second; the model adjudicates.

   No bypass. Two cheap signals as priors, one model deciding, so nothing executes unreasoned.

3. **Plan** — decompose into steps, each declaring its action, arguments, *and an explicit test for what success looks like*.

   The success test is what would have caught the heritage report: “a file exists containing named sites” is not satisfied by the analytics agent returning cleanly.

4. **Safety gate** — an independent review of the plan before anything runs; irreversible actions require confirmation.

   Exists today and works. It moves earlier, so it sees a whole plan rather than one step.

5. **Execute, one step at a time** — dispatch deterministically, verify the effect occurred, then verify it against the step's success test. Append the observation either way.

   Deterministic dispatch is retained deliberately: the model chooses what to do, never carries it out.

6. **Reflect** — was the *goal* met, not merely each step. If not and budget remains, replan from the current state.

   Step-level success summing to goal-level failure is the most common way this system misleads its user.

7. **Respond** — written from the same context that did the work, including what failed.

8. **Learn** — record instruction, plan, and outcome as a labelled example.

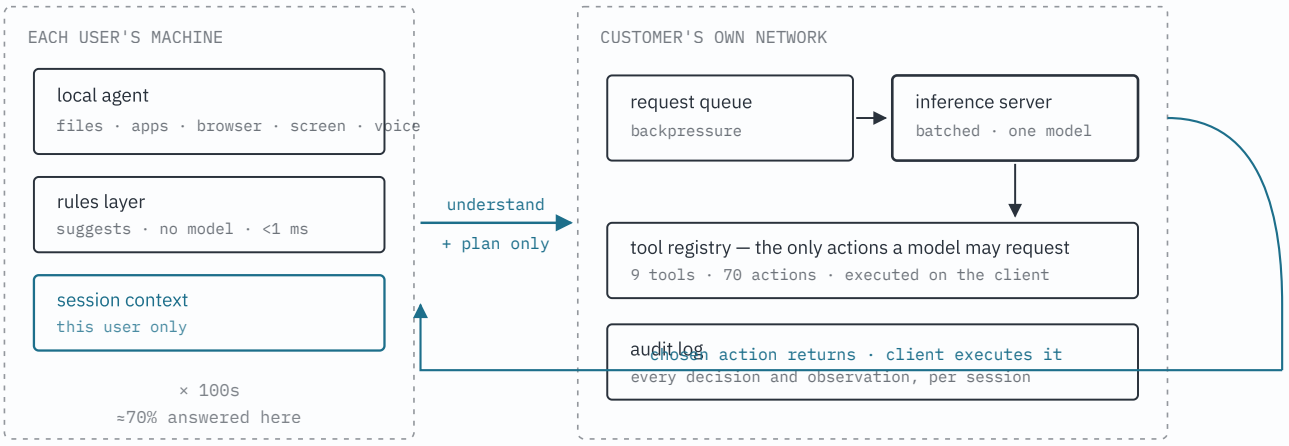
   Currently the learning subsystem influences 0% of routing decisions, measured over 400 commands. Feeding the classifier is the only place that signal can change behaviour.

**What makes it robust rather than merely elegant**

| PROPERTY | MECHANISM |
|---|---|
| `termination` | Per-turn budget in steps and seconds. Repeating an action with identical arguments is detected and stops the loop. |
| `correctness` | Two verifications per step: the effect occurred, and it satisfies the step's declared success test. |
| `recoverability` | A failed step replans from observed state rather than retrying blindly; bounded attempts. |
| `honesty` | An unverifiable effect is reported as unverified, never as success. Enforced by a standing audit over the source. |
| `safety` | Plan-level review before execution; irreversible actions gated on explicit confirmation. |
| `observability` | The context object is the trace — every decision, action and observation of a turn, inspectable after the fact. |

## 04 Scaling past a single machine

The requirement is hundreds of concurrent users with data never leaving the customer's network. Those are compatible, because the work splits cleanly.




> Only the understand and plan stages cross the network. Actions execute on the machine that owns the files, so the server never touches user data — and because the rules layer answers roughly seven instructions in ten locally, a hundred users generate far less inference load than a hundred would suggest.

Three properties follow from that split, and they are the ones worth checking hardest in review:

- **The server is stateless per request.** Session context lives with the client and is sent as part of the request. Any server instance can answer any turn, so scaling is horizontal.
- **Concurrency is bounded by the queue, not by the model.** Batched inference serves many sequences at once; the queue applies backpressure rather than letting latency degrade silently.
- **Privacy is structural, not promised.** Documents and credentials never leave the client. What crosses the network is an instruction and a context summary, inside the customer's own network.

## 05 Where this is honestly weak

The design's viability rests on one thing that has been measured and does not currently hold.

THE BINDING CONSTRAINT

Every stage above assumes the model can reliably choose an action and its arguments from a set of 70. **It cannot, at the model size that fits the current hardware.** Asked to count tasks and save the number to a file, the existing reasoning loop selected the analytics tool three times with identical arguments and stopped. Compound instructions currently succeed in 8 of 18 benchmark cases, and that ratio is a tool-selection problem, not an architecture problem.

A per-step accuracy of 90% is a 59% success rate across five steps. The loop above is the correct shape and will not rescue a model that picks the wrong tool — which makes tool-selection accuracy as a function of model size the measurement that should decide both the architecture and the hardware, and it is the next thing being built.

## 06 Questions for review

> **Is per-step success criteria the right place for correctness?**
>
> The alternative is a single goal-level check at the end — cheaper, but it cannot tell which step went wrong, so it cannot replan intelligently.

**Where should session context live at scale?**

Proposed: with the client, sent per request, keeping the server stateless. That grows the request payload with conversation length and needs a truncation policy. A server-side store is the obvious alternative and reintroduces state.

**Is one shared model the right call for hundreds of users?**

Batching makes it efficient, but it is a single failure domain and one queue. Per-team instances cost more and isolate failure.

**Should the rules layer survive at all once tool-selection is reliable?**

It is 95% accurate and effectively free, but it is hand-written and does not generalise. The argument for keeping it is latency; the argument against is that two systems must agree on what an instruction means.

---

VEGA · Autonomous System Controller · 526 tests · figures measured on the development machine, September 2026. This document describes the control loop only; capability inventory and security model are covered separately.